Typescript In Practice

4C. TypeScript in Practice

Questions: 9 **Difficulty:** Mid to Senior **Companies:** Google, Amazon, Flipkart, Razorpay, Swiggy, Microsoft, Atlassian

These questions test whether you can apply TypeScript to real frontend codebases. Knowing generics in theory means nothing if you can't type a React component or an API response properly.

Q 4.17 How do you type React components — props, events, refs, children, and generic components?

Difficulty: Mid **Asked at:** Every company

Basic props typing:

TYPESCRIPT Copy

```typescript
interface ButtonProps {
  label: string;
  onClick: () => void;
  variant?: "primary" | "secondary" | "danger";
  disabled?: boolean;
}

function Button({ label, onClick, variant = "primary", disabled = false }:
  return (
    <button className={variant} onClick={onClick} disabled={disabled}>
```

```
}
```

Typing events:

TYPESCRIPT Copy

```typescript
// Specific event types — not generic 'any'
function SearchInput() {
  const handleChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    console.log(e.target.value);
  };

  const handleSubmit = (e: React.FormEvent<HTMLFormElement>) => {
    e.preventDefault();
  };

  const handleKeyDown = (e: React.KeyboardEvent<HTMLInputElement>) => {
    if (e.key === "Enter") { /* ... */ }
  };

  return (
    <form onSubmit={handleSubmit}>
      <input onChange={handleChange} onKeyDown={handleKeyDown} />
    </form>
  );
}
```

Typing children:

TYPESCRIPT Copy

```typescript
// When children can be anything renderable
interface CardProps {
  children: React.ReactNode;  // strings, numbers, elements, arrays, fragm
  title: string;
}

// When children must be a specific type
```

```
interface TabsProps {
  children: ReactElement<TabProps>[];  // only Tab elements
}

// When children is a render function
interface DataFetcherProps<T> {
  url: string;
  children: (data: T, loading: boolean) => React.ReactNode;
}
```

Typing refs:

TYPESCRIPT Copy

```
function TextInput() {
  const inputRef = useRef<HTMLInputElement>(null);

  const focus = () => {
    inputRef.current?.focus();
  };

  return <input ref={inputRef} />;
}

// Ref for a custom component using forwardRef
const FancyInput = forwardRef<HTMLInputElement, FancyInputProps>(
  ({ label, ...props }, ref) => {
    return (
      <div>
        <label>{label}</label>
        <input ref={ref} {...props} />
      </div>
    );
  }
);
```

Generic components:

TYPESCRIPT Copy

```
  value: T;
  onChange: (value: T) => void;
  getLabel: (item: T) => string;
  getKey: (item: T) => string;
}

function Select<T>({ options, value, onChange, getLabel, getKey }: SelectP
  return (
    <select
      value={getKey(value)}
      onChange={(e) => {
        const selected = options.find(opt => getKey(opt) === e.target.valu
        if (selected) onChange(selected);
      }}
    >
      {options.map(option => (
        <option key={getKey(option)} value={getKey(option)}>
          {getLabel(option)}
        </option>
      ))}
    </select>
  );
}

// Usage — TypeScript infers T from the options array
<Select
  options={countries}
  value={selectedCountry}
  onChange={setSelectedCountry}
  getLabel={(c) => c.name}     // c is typed as Country
  getKey={(c) => c.code}
/>
```

Common mistake: Using `React.FC` unnecessarily. It adds implicit `children` to every component (removed in React 18 types, but still confusing), and makes generic components harder to type. Just type the props directly in the function parameters.

Q 4.18 How do you type custom hooks?

Difficulty: Mid **Asked at:** Flipkart, Swiggy, Razorpay, Amazon

Custom hooks are just functions that use other hooks. Type them like any other function — parameters and return values:

TYPESCRIPT Copy

```typescript
// Simple hook — TypeScript infers the return type
function useToggle(initialValue = false) {
  const [value, setValue] = useState(initialValue);

  const toggle = useCallback(() => setValue(v => !v), []);
  const setTrue = useCallback(() => setValue(true), []);
  const setFalse = useCallback(() => setValue(false), []);

  return { value, toggle, setTrue, setFalse };
}

// Usage
const { value: isOpen, toggle, setFalse: close } = useToggle();
```

Hook with explicit return type (for complex cases):

TYPESCRIPT Copy

```typescript
interface UseFetchResult<T> {
  data: T | null;
  error: Error | null;
  loading: boolean;
  refetch: () => void;
}

function useFetch<T>(url: string): UseFetchResult<T> {
  const [data, setData] = useState<T | null>(null);
  const [error, setError] = useState<Error | null>(null);
  const [loading, setLoading] = useState(true);

  const fetchData = useCallback(async () => {
    setLoading(true);
    setError(null);
```

```
      const json = await response.json();
      setData(json as T);
    } catch (err) {
      setError(err instanceof Error ? err : new Error(String(err)));
    } finally {
      setLoading(false);
    }
  }, [url]);

  useEffect(() => { fetchData(); }, [fetchData]);

  return { data, error, loading, refetch: fetchData };
}

// Usage — specify the expected data type
const { data, loading, error } = useFetch<User[]>("/api/users");
// data is User[] | null
```

Hook that returns a tuple (like useState):

TYPESCRIPT Copy

```
function useLocalStorage<T>(key: string, initialValue: T): [T, (value: T)
  const [storedValue, setStoredValue] = useState<T>(() => {
    try {
      const item = localStorage.getItem(key);
      return item ? JSON.parse(item) : initialValue;
    } catch {
      return initialValue;
    }
  });

  const setValue = (value: T) => {
    setStoredValue(value);
    localStorage.setItem(key, JSON.stringify(value));
  };

  return [storedValue, setValue];
}
```

Follow-up: "When should you explicitly type the return value vs let inference handle it?"

Explicitly type the return value when the hook is exported and used across many files. This makes the API contract clear and prevents accidental breaking changes. Let inference handle it for hooks that are local to a single component or module.

Q 4.19 How do you type API responses? What's runtime validation with Zod?

Difficulty: Senior **Asked at:** Razorpay, Google, Cred, Flipkart

The problem: TypeScript types disappear at runtime. When you get data from an API, TypeScript has no idea if the actual response matches your type definition. You're trusting the backend blindly.

TYPESCRIPT Copy

```typescript
interface User {
  id: string;
  name: string;
  email: string;
}

// TypeScript trusts this cast — but what if the API returns something els
const response = await fetch("/api/user/1");
const user = await response.json() as User;
// If the API returns { id: 123, fullName: "Ankit" }, TypeScript won't cat
// You'll get runtime errors when you access user.name (undefined)
```

The solution: runtime validation with type inference

Libraries like Zod, Valibot, and ArkType let you define a schema that validates data at runtime and infers the TypeScript type from the same definition. One source of truth, no duplication.

TYPESCRIPT

Copy

```typescript
import { z } from "zod";

// Define the schema — this is both the runtime validator AND the type def
const UserSchema = z.object({
  id: z.string(),
  name: z.string(),
  email: z.string().email(),
  age: z.number().min(0).optional(),
  role: z.enum(["admin", "editor", "viewer"]),
  createdAt: z.string().datetime().transform(str => new Date(str)),
});

// Infer the TypeScript type from the schema — no need to write it manuall
type User = z.infer<typeof UserSchema>;
// { id: string; name: string; email: string; age?: number; role: "admin"

// Safe API call with runtime validation
async function fetchUser(id: string): Promise<User> {
  const response = await fetch(`/api/users/${id}`);
  const data = await response.json();

  // This throws a ZodError if the data doesn't match the schema
  const user = UserSchema.parse(data);
  return user;
}

// Or use safeParse for non-throwing validation
async function fetchUserSafe(id: string) {
  const response = await fetch(`/api/users/${id}`);
  const data = await response.json();

  const result = UserSchema.safeParse(data);

  if (!result.success) {
    console.error("Validation failed:", result.error.issues);
    return null;
  }
```

```
// Properly typed as User
}
```

With React Query / TanStack Query:

TYPESCRIPT Copy

```typescript
function useUser(id: string) {
  return useQuery({
    queryKey: ["user", id],
    queryFn: async () => {
      const response = await fetch(`/api/users/${id}`);
      const data = await response.json();
      return UserSchema.parse(data);  // Validated and typed
    },
  });
}
```

Why interviewers ask this: It tests whether you understand the boundary between TypeScript's compile-time safety and runtime reality. Strong candidates know that type assertions (`as User`) are trust-based and can fail silently, while runtime validation catches issues at the boundary where external data enters your application.

Q 4.20 How do you type state management stores — Redux/Zustand?

Difficulty: Mid **Asked at:** Flipkart, Swiggy, Amazon, Walmart

Zustand — the simpler case:

TYPESCRIPT Copy

```typescript
import { create } from "zustand";

interface CartItem {
  id: string;
```

```
  name: string;
  quantity: number;
}

interface CartStore {
  items: CartItem[];
  total: number;
  addItem: (item: Omit<CartItem, "quantity">) => void;
  removeItem: (id: string) => void;
  updateQuantity: (id: string, quantity: number) => void;
  clearCart: () => void;
}

const useCartStore = create<CartStore>((set, get) => ({
  items: [],
  total: 0,

  addItem: (item) =>
    set((state) => {
      const existing = state.items.find(i => i.id === item.id);
      if (existing) {
        return {
          items: state.items.map(i =>
            i.id === item.id ? { ...i, quantity: i.quantity + 1 } : i
          ),
        };
      }
      return { items: [...state.items, { ...item, quantity: 1 }] };
    }),

  removeItem: (id) =>
    set((state) => ({
      items: state.items.filter(i => i.id !== id),
    })),

  updateQuantity: (id, quantity) =>
    set((state) => ({
      items: state.items.map(i =>
        i.id === id ? { ...i, quantity } : i
      ),
    })),

  clearCart: () => set({ items: [], total: 0 }),
```

```
// Usage — fully typed
const items = useCartStore(state => state.items);
const addItem = useCartStore(state => state.addItem);
```

Redux Toolkit — typed dispatch and selectors:

TYPESCRIPT Copy

```typescript
import { configureStore, createSlice, PayloadAction } from "@reduxjs/toolk

interface CounterState {
  value: number;
  status: "idle" | "loading" | "failed";
}

const initialState: CounterState = { value: 0, status: "idle" };

const counterSlice = createSlice({
  name: "counter",
  initialState,
  reducers: {
    increment: (state) => { state.value += 1; },
    decrement: (state) => { state.value -= 1; },
    incrementByAmount: (state, action: PayloadAction<number>) => {
      state.value += action.payload;
    },
  },
});

const store = configureStore({
  reducer: { counter: counterSlice.reducer },
});

// Infer types from the store itself — single source of truth
type RootState = ReturnType<typeof store.getState>;
type AppDispatch = typeof store.dispatch;

// Typed hooks — use these instead of plain useSelector/useDispatch
import { useSelector, useDispatch } from "react-redux";

const useAppDispatch = useDispatch.withTypes<AppDispatch>();
```

```
const useAppSelector = useSelector.withTypes<RootState>();
// Usage
function Counter() {
  const count = useAppSelector(state => state.counter.value);  // typed as
  const dispatch = useAppDispatch();

  dispatch(counterSlice.actions.incrementByAmount(5));  // payload typed a
  dispatch(counterSlice.actions.incrementByAmount("5")); // Error: string
}
```

The key pattern: derive `RootState` and `AppDispatch` from the store using `ReturnType` and `typeof`. Never define these types manually — they'll go out of sync when you add new slices.

Q 4.21 How do you type higher-order components and render props?

Difficulty: Senior **Asked at:** Google, Microsoft, Amazon

Higher-order components (HOCs) are functions that take a component and return a new component with additional props. Typing them correctly is notoriously tricky.

TYPESCRIPT Copy

```
// HOC that adds loading behavior
function withLoading<P extends object>(
  WrappedComponent: React.ComponentType<P>
) {
  interface WithLoadingProps {
    isLoading: boolean;
  }

  function WithLoadingComponent(props: P & WithLoadingProps) {
    const { isLoading, ...componentProps } = props;

    if (isLoading) return <div>Loading...</div>;
```

```
  WithLoadingComponent.displayName =
    `WithLoading(${WrappedComponent.displayName || WrappedComponent.name})

  return WithLoadingComponent;
}

// Usage
interface UserListProps {
  users: User[];
}

const UserList = ({ users }: UserListProps) => (
  <ul>{users.map(u => <li key={u.id}>{u.name}</li>)}</ul>
);

const UserListWithLoading = withLoading(UserList);

// TypeScript knows this needs both UserListProps AND { isLoading: boolean
<UserListWithLoading users={users} isLoading={false} />
```

Render props pattern:

TYPESCRIPT Copy

```
interface MouseTrackerProps {
  children: (position: { x: number; y: number }) => React.ReactNode;
}

function MouseTracker({ children }: MouseTrackerProps) {
  const [position, setPosition] = useState({ x: 0, y: 0 });

  useEffect(() => {
    const handler = (e: MouseEvent) => setPosition({ x: e.clientX, y: e.cl
    window.addEventListener("mousemove", handler);
    return () => window.removeEventListener("mousemove", handler);
  }, []);

  return <>{children(position)}</>;
}
```

```
<MouseTracker>
  {({ x, y }) => <div>Mouse: {x}, {y}</div>}
</MouseTracker>
```

The honest take: HOCs are becoming less common in modern React. Custom hooks solve the same problem with simpler types. But interviewers still ask about HOCs because they test your understanding of generics and type propagation. If asked "which pattern would you prefer?", the answer is custom hooks — they're easier to type, compose, and test.

Q 4.22 What are `.d.ts` declaration files? When do you write your own?

Difficulty: Mid **Asked at:** Amazon, Microsoft, Flipkart

Declaration files (`.d.ts`) describe the types for JavaScript code that doesn't have its own TypeScript types. They contain only type information — no runtime code.

Where declaration files come from:

1. **DefinitelyTyped** — the `@types/*` packages on npm. When you `npm install @types/lodash`, you get `lodash/index.d.ts` with all the type definitions.
2. **The TypeScript compiler** — when you set `"declaration": true` in tsconfig, the compiler generates `.d.ts` files alongside your JavaScript output. This is how TypeScript libraries ship types.
3. **You write them manually** — for untyped third-party libraries, global variables, or non-JS assets.

Common cases where you need to write your own:

TYPESCRIPT Copy

```
// styles.d.ts
declare module "*.module.css" {
  const classes: Record<string, string>;
  export default classes;
}

// 2. Typing image imports
// assets.d.ts
declare module "*.png" {
  const src: string;
  export default src;
}

declare module "*.svg" {
  const content: React.FC<React.SVGProps<SVGSVGElement>>;
  export default content;
}

// 3. Typing environment variables
// env.d.ts
declare namespace NodeJS {
  interface ProcessEnv {
    NEXT_PUBLIC_API_URL: string;
    DATABASE_URL: string;
    JWT_SECRET: string;
  }
}

// 4. Typing a third-party library that has no types
// untyped-lib.d.ts
declare module "some-untyped-library" {
  export function doSomething(input: string): Promise<Result>;
  export interface Result {
    data: unknown;
    status: number;
  }
}

// 5. Typing global variables (analytics, feature flags)
// globals.d.ts
declare global {
  interface Window {
```

```
}
export {};  // Makes this file a module
```

Follow-up: "What's the difference between `declare module` and `declare global`?"

`declare module "name"` augments or defines types for a specific npm module. `declare global` augments the global scope (Window, globalThis, NodeJS.ProcessEnv). The `export {}` at the end of a file with `declare global` is necessary to make the file a module — without it, TypeScript treats the file as a script and the global augmentation doesn't work correctly.

Q 4.23 Walk through the important `tsconfig.json` options.

Difficulty: Mid **Asked at:** Razorpay, Google, Amazon

You don't need to memorize every option, but interviewers expect you to know the ones that affect code quality and project structure:

JSONC Copy

```
{
  "compilerOptions": {
    // ═══ Strictness ═══
    "strict": true,                // Enables all strict checks (recommended
    // Includes: strictNullChecks, noImplicitAny, strictFunctionTypes,
    // strictBindCallApply, strictPropertyInitialization, noImplicitThis,

    // ═══ Module System ═══
    "module": "esnext",            // Output ESM (import/export)
    "moduleResolution": "bundler", // Modern resolution for Vite/Next.js/w
    "esModuleInterop": true,       // Allows default imports from CommonJS m
    "isolatedModules": true,       // Required for Babel/SWC/esbuild transpi

    // ═══ Output ═══
    "target": "es2022",            // JS version to output - match your brow
```

```
    // == Path Resolution ==
    "baseUrl": ".",
    "paths": {
      "@/*": ["src/*"],          // Import as @/components/Button instead
      "@utils/*": ["src/utils/*"]
    },

    // == Quality Checks ==
    "noUnusedLocals": true,      // Error on unused variables
    "noUnusedParameters": true,  // Error on unused function parameters
    "noFallthroughCasesInSwitch": true,  // Error on switch fallthrough
    "forceConsistentCasingInFileNames": true,  // Catch case-sensitive imp

    // == Other ==
    "skipLibCheck": true,        // Skip type checking .d.ts files (faster
    "resolveJsonModule": true    // Allow importing .json files
  },
  "include": ["src"],
  "exclude": ["node_modules", "dist"]
}
```

The ones that matter most in interviews:

- `strict: true` — Non-negotiable. If someone asks "what's the first thing you'd set in tsconfig?", this is it.
- `moduleResolution: "bundler"` — The modern setting for projects using Vite, Next.js, or webpack. Replaces the older `"node"` resolution.
- `isolatedModules: true` — Required when your build tool (Vite, SWC, esbuild) transpiles files individually rather than as a whole program. Catches patterns that only work with the full TypeScript compiler.
- `paths` — Clean imports instead of relative path chains. Essential for projects with deep folder structures.

Follow-up: "What's the difference between `moduleResolution: 'node'` and `'bundler'`?"

imports — it understands `exports`, doesn't require file extensions, and handles ESM/CJS interop correctly. For any new project using Vite, Next.js, or similar tools, use `"bundler"`.

Q 4.24 What are common TypeScript mistakes you've seen in real codebases?

Difficulty: Mid **Asked at:** Google, Razorpay, Flipkart, Atlassian

This is a behavioral-meets-technical question. Interviewers want to hear real examples, not textbook theory.

Mistake 1: Using `any` to silence errors

TYPESCRIPT Copy

```typescript
// The codebase is full of this
const data: any = await fetchData();
data.whatever.you.want;  // No errors, no safety, no point using TypeScrip

// Fix: Use unknown and narrow
const data: unknown = await fetchData();
if (isUser(data)) {
  data.name;  // Safe after narrowing
}
```

Mistake 2: Over-typing — annotating what TypeScript already knows

TYPESCRIPT Copy

```typescript
// Unnecessary annotations everywhere
const name: string = "Ankit";
const items: string[] = ["a", "b", "c"];
const doubled: number = items.length * 2;
```

```
// let inference do its job
const items = ["a", "b", "c"];
const doubled = items.length * 2;
```

Mistake 3: Using type assertions instead of type guards

TYPESCRIPT Copy

```
// Dangerous — TypeScript trusts you without checking
const user = response.data as User;

// Safe — runtime check that TypeScript understands
if (isUser(response.data)) {
  const user = response.data;  // Properly narrowed
}
```

Mistake 4: Not using discriminated unions for state

TYPESCRIPT Copy

```
// Bad — impossible states are representable
interface State {
  loading: boolean;
  error: string | null;
  data: User[] | null;
}
// Can have loading: true AND error: "failed" AND data: [...] simultaneous

// Good — each state is distinct and complete
type State =
  | { status: "idle" }
  | { status: "loading" }
  | { status: "error"; message: string }
  | { status: "success"; data: User[] };
```

Mistake 5: Using `interface` for everything, including unions

TYPESCRIPT Copy

```
  // How do you say "either success with data OR failure with error"?
}

// Types handle this naturally
type Result<T> =
  | { ok: true; data: T }
  | { ok: false; error: Error };
```

Mistake 6: Not typing errors in catch blocks

TYPESCRIPT Copy

```
// error is 'unknown' in TypeScript 4.4+ — you can't access .message direc
try {
  await fetchData();
} catch (error) {
  console.log(error.message);  // Error: 'error' is of type 'unknown'

  // Correct
  if (error instanceof Error) {
    console.log(error.message);
  }
}
```

Q 4.25 How do you migrate a JavaScript codebase to TypeScript incrementally?

Difficulty: Senior **Asked at:** Amazon, Flipkart, Razorpay, Walmart

This is a process question, not a code question. Interviewers want to hear that you've done this (or at least thought deeply about it) and understand the practical challenges.

Step 1: Set up TypeScript alongside JavaScript

JSONC Copy

```
{
  "compilerOptions": {
    "allowJs": true,              // Let .js files coexist with .ts files
    "checkJs": false,             // Don't type-check .js files yet
    "strict": false,              // Start without strict mode
    "noEmit": true,               // Let your bundler handle compilation
    "jsx": "react-jsx",
    "moduleResolution": "bundler",
    "esModuleInterop": true,
    "isolatedModules": true
  },
  "include": ["src"]
}
```

Step 2: Rename files gradually — start from the leaves

Start with utility files, constants, and types that don't import many other modules. These are the "leaves" of your dependency tree — they have the fewest connections:

JAVASCRIPT Copy

```
src/
  utils/
    formatDate.js   →  formatDate.ts    (rename first — no dependencies)
    constants.js    →  constants.ts
  types/
    user.ts                              (create new type files)
    api.ts
  components/
    Button.jsx      →  Button.tsx        (rename later — depends on other f
    Dashboard.jsx                        (rename last — imports many things
```

Step 3: Add types incrementally

Don't try to make everything perfectly typed on the first pass. Use `// @ts-ignore` or temporary `any` for complex cases and come back to them:

TYPESCRIPT Copy

```
  // ...
}

// Second pass — add proper types
interface Order {
  id: string;
  items: OrderItem[];
  total: number;
}

export function processOrder(order: Order) {
  // ...
}
```

Step 4: Enable strict mode progressively

JSONC Copy

```
// Phase 1: Start here
"strict": false

// Phase 2: Enable one check at a time
"noImplicitAny": true        // Fix all implicit any types
// Then:
"strictNullChecks": true     // Fix all null/undefined handling
// Then:
"strictFunctionTypes": true  // Fix function type compatibility

// Phase 3: Enable full strict
"strict": true
```

Each check you enable will surface a batch of errors. Fix them one batch at a time. Don't turn on `strict: true` on day one in a large codebase — you'll get thousands of errors and nobody will want to work on it.

Step 5: Add CI checks

YAML Copy

```
- name: Check for .js files
  run: |
    if find src -name "*.js" -o -name "*.jsx" | grep -q .; then
      echo "Found .js files — please convert to TypeScript"
      exit 1
    fi
```

How long does it take? For a medium-sized codebase (200-500 files), expect 2-4 weeks for a small team working on it part-time. The first 80% goes fast. The last 20% (complex generics, third-party library types, legacy patterns) takes disproportionately longer.

What interviewers want to hear: That you understand this is a gradual process, not a rewrite. That you start from the leaves and work inward. That you enable strict checks one at a time. And that you've thought about how to prevent regression (CI checks, no new .js files).

QUICK CHECK — no marks, just for you.

1. You write `const user = await response.json() as User;` and the API actually returns `{ id: 123, fullName: "Ankit" }`. According to the lesson, what happens?

- ○ TypeScript accepts the cast at compile time, and you only discover the mismatch as a runtime error when you read user.name
- ○ response.json() throws immediately because the parsed JSON is not assignable to the declared User type
- ○ TypeScript strips the extra fullName key and fills missing properties of User with undefined at runtime
- ○ The compiler rejects the cast because the returned shape does not structurally match the User interface

2. Per the lesson's guidance, when should a custom hook get an explicit return type annotation rather than relying on inference?

- ◯ Always — inference on hook return values is unsupported by TypeScript
- ◯ Only when the hook returns a tuple, because tuples are otherwise inferred as arrays of a union type
- ◯ Whenever the hook calls useState, since useState's inferred type is unreliable
- ◯ When the hook is exported and used across many files, so the API contract is clear and breaking changes are prevented

Check answer

3. In the Redux Toolkit example, RootState is written as `ReturnType<typeof store.getState>` instead of a hand-written interface. What is the lesson's reason for this?

- ◯ ReturnType makes the state type mutable, which Immer requires inside createSlice reducers
- ◯ A manually written state type will drift out of sync with the store as new slices are added
- ◯ useSelector only accepts types produced by ReturnType and rejects plain interfaces
- ◯ Hand-written interfaces cannot describe the nested reducer keys that configureStore produces

Check answer

Generics Advanced Types
Next
Networking And Protocols

LEARN MORE

ARTICLES & DOCS

- React + TypeScript Cheatsheet
  github.com

VIDEOS



React & TypeScript best practices
YouTube



All the TypeScript you need for React
YouTube



React + TypeScript best practices (beginners)
YouTube

Hand-picked references for this lesson.